

DeepTutor 小白指南

港大开源的 AI 家教：不是聊天框，而是一个“能力运行时”——还能把你本机的 Claude Code 叫进来一起讲题

HKUDS · 港大数据智能实验室

v1.6.12 · 40k★

Python 后端 + Next.js 前端

本机实测通过 ✓

一句话看懂：DeepTutor 把“讲题、出题、研究、画图、精读、精看视频”这些学习能力全部跑在 **同一个智能体循环** 上，配三层长期记忆和可插拔 RAG 引擎，是一个装在你本机上的 **完整学习系统**。最特别的是：它能把本机的 Claude Code / Codex 等编程智能体 **当作工具调用**，让“会动手的管家”给“会讲题的老师”打工。

Q1

DeepTutor 到底是什么？和“把 PDF 拖进 ChatGPT”有什么本质区别？

普通做法：问一句答一句。关掉窗口，AI 不记得你卡在哪一章、哪个知识点已经会了，下次重开还得从头讲。

DeepTutor 的做法：它是一个跑在你本机的 **Web 应用**（两条命令启动），里面住着一个 **有长期记忆的导师智能体**——你上传的教材变成可检索的知识库，你的每次提问和纠错被记成学习轨迹，它据此调整讲法。它没有把功能做成一个聊天框，而是做成了一个 **能力运行时**：讲题、出题、写报告、画图、沉浸阅读都在同一个引擎上运转。

维度	网页聊天框	DeepTutor
记忆	会话结束即忘	三层记忆，跨会话累积“你是谁、学到哪、偏好什么”
教材	塞上下文，塞不下就截断	建成知识库，按需检索引用到页码
能力	只有文本对话	出题/批改/研究/画图/精读/视频学习共享一个引擎
数据	在别人服务器上	全部在本机 data/ 目录

聊天框模式

一问一答 · 会话即弃

VS

DeepTutor 能力运行时

讲题

出题批改

研究/画图

三层长期记忆

可插拔 RAG

Q2

"Agent-Native (智能体原生)"是什么意思? 它和普通 AI 应用的架构差别在哪?

v1.0 重写时 (约 20 万行代码), 作者做了一个关键决定: **所有学习功能不再各写各的流水线, 而是全部压到同一个智能体循环上**。你在知识库里的学 **[[Agent循环]]**——"模型读消息列表 → 要么回答、要么调工具 → 工具结果回填 → 再问模型"——就是整个系统的唯一心脏。

"原生"指的是: 每个功能 = 一个能力 (Capability) + 一组工具 + 一段提示词, 挂到循环上即生效。新功能不用重写架构, 写个插件就行。这和"套壳聊天应用" (一个输入框 + 一条 API 转发) 是两个物种。



类比: 普通应用像每道菜一个专用厨房; DeepTutor 是一个中央厨房, 菜谱 (能力) 插上就能做。

Q3

装完之后, 我的电脑上到底跑了什么?

架构地图

`pip install deeptutor + python -m deeptutor start` 之后, 本机会有两个服务进程:

浏览器 → 前端 Next.js (端口 3782) → 后端 FastAPI (端口 8001) → LLM / Ollama / 搜索引擎
↓
F:\reminder\deeptutor\data\ ← 你的全部数据: 配置、记忆、知识库、产出

关键是 **data/user/settings/** 下那堆纯 JSON/YAML 配置文件——`model_catalog.json` (模型和 key)、`system.json` (端口)、`subagent.json` (子智能体预算) 等。网页上的 Settings 页面本质就是这些文件的编辑器, 所以**直接改文件 = 直接改配置**, 没有隐藏状态。



Q4

"能力 (Capability)" 到底是什么？十种模式为什么能共享一个运行时？

打开源码 `deeptutor/capabilities/`，每个文件夹就是一个能力：`ask_questions`、`solve`、`mastery`（`mastery path` 闯关）、`reading`、`watching`、`subagent`..... 每个能力 = 三件套：

能力 = 协议(`protocol.py`)：我需要哪些工具、哪些上下文（知识库/记忆/工作区）
 + 提示词：这个模式下导师的人设和规则
 + 工具挂载：运行时按需把工具装到循环上（比如选了知识库才挂 `rag` 工具）

这在你知识库能找到影子：[[构建流水线与Pass]] 说"复杂工作拆成顺序阶段"，DeepTutor 的做法更近一步——阶段不是写死的，而是"能力"这个标准件，由注册表 (`registry.py`) 统一管理，像 USB 接口一样即插即用。

实测体验：同一个会话里，讲完题直接"帮我出 5 道题"再"把这道题画成图"，导师不用重新自我介绍、记忆不丢——因为切换的只是挂载的工具和提示词，循环本身没换。

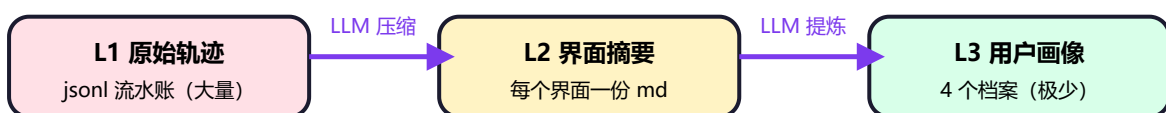
Q5

三层记忆 L1 / L2 / L3 是怎么工作的？为什么不是把聊天记录全存下来？

全存下来有两个致命问题：上下文塞不下、噪音淹没重点。DeepTutor 的方案像"日记 → 周记 → 个人简历"：最有特色的设计

层	是什么	存在哪（源码实测）	类比
L1 轨迹层	原始事件流水账：每次提问/检索/纠错	<code>trace/<界面>/<日期>.jsonl</code> 只追加	监控日志
L2 表层摘要	每个使用界面（ <code>chat/kb/quiz...</code> ）一份摘要	<code>L2/<界面>.md</code> 带脚注引用	周记
L3 综合层	跨界面浓缩成 4 个档案：近况/画像/范围/偏好	<code>L3/recent profile scope preferences.md</code>	个人简历

由 **consolidator (LLM 驱动)** 定期把 L1 压缩成 L2、L2 提炼成 L3；Memory Graph 把三层互相链接——**L3 里每一句"你总忘记初始化列表"都能点回 L1 的原始证据**，个性化可见、可编辑，不是黑箱。



越往右：体积越小、密度越高、注入提示词的成本越低

Q6

它的知识库 / RAG，和我在 langchain 里学的有什么不同？

对比已学

基础原理一模一样（[[RAG检索增强生成]]：切块 → 嵌入成向量 → 最近邻检索 → 塞进提示词），DeepTutor 的增量在三处：

- ① **引擎可插拔**：同一份文档可以选不同检索引擎建库——**LlamaIndex**（经典向量检索，本机实测默认）、**GraphRAG**（微软的知识图谱检索，适合“全局问题”）、**LightRAG**（港大自家的轻量图 RAG）、**PageIndex**（把文档建成决策树，让模型像翻目录一样推理检索）、还能直连腾讯 IMA、MarginNote 4、Obsidian 库、Kiwix 离线维基包。
- ② **索引版本化**：换嵌入模型后索引不会悄悄失配——每个索引记录“它是谁、用什么建的”，换引擎要你确认重建，杜绝“向量坐标系对不上还硬查”的假结果。
- ③ **解析引擎也可插拔**：PDF 怎么切块（MinerU / Docling / PyMuPDF4LLM / Apache Tika...）与检索分开选。

和 langchain 实验的差别：langchain 里你手写 `VectorStore.from_documents()`；这里全被封装成“建库向导 + 命令行”。原理没变，**工程化程度不同**——这正是读生产级开源项目的价值。

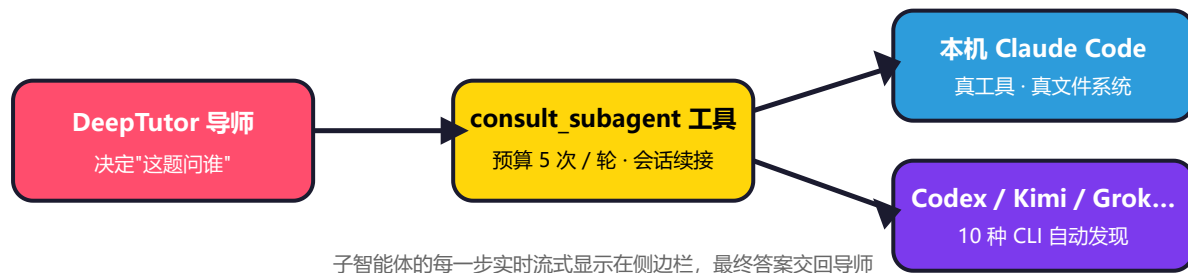
Q7

它怎么把我本机的 Claude Code “叫进来”当助手？机制是什么？

推文最大亮点

这是全项目最巧的一招，源码就三个文件（capabilities/subagent/，约 600 行）：

- ① **自动发现**：启动时扫描你机器的 PATH，认出已装的编程智能体 CLI——Claude Code、Codex、Grok CLI、Kimi CLI、opencode、Gemini 等 10 种，全部列在 Settings → My Agents 里；
- ② **工具化**：一旦你在聊天中选了某个子智能体，运行时往循环上挂一个 `consult_subagent` 工具。导师模型决定“这个问题该问谁”，填一个 `question` 参数调用它；
- ③ **无头运行 + 流式回传**：DeepTutor 以**非交互模式**把 CLI 拉起来（Claude Code 走 `--permission-mode bypassPermissions`，绝不卡在“是否允许”的确认上），子智能体的每一步工具调用**实时流式显示在侧边栏**，最终答案回到导师手里；
- ④ **咨询预算**：默认一轮最多咨询 5 次（subagent.json 里 `consult_budget`，可调 1~12），同一轮内多次咨询共享同一个子智能体会话，上下文连续。预算一用完，工具直接拒绝并“勒令”导师自己作答。



一句话：导师是**会讲题的老师**，子智能体是**会动手的管家**——老师负责判断“该不该问、问什么”，管家负责真跑真查。

Q8

子智能体和"导师自己回答"有什么区别？为什么权限边界这么重要？

安全边界

导师（LLM）只会**生成文字**；子智能体（Claude Code）在你的机器上**真的执行**——读文件、跑命令、改代码。权利越大，边界越严，DeepTutor 的沙盒设计：

- **内容工作区（Content Workspace）**：智能体被要求只读工作区、只往 `outputs/能力/会话/轮次/` 写文件；工作区外想复制文件要用户点"Allow once"；
- **执行沙盒**：优先走 Docker 隔离容器 / Linux bubblewrap，都没有才降级为"受限子进程"（Settings 里明确标着 *best effort*）；
- **密钥隔离**：API key 存在智能体够不到的 sandbox 作用域外。

本机实测撞上的活例子：让导师咨询 Claude Code"数一下 F 盘某目录有几个文件"，子智能体老老实实回答"那个路径不在我的工作区里，我够不到"——**边界真的拦住了它**，而不是嘴上说说的安全。想让它看到，把材料放进工作区即可。

Q9

Skills、MCP、CLI Apps 这三种扩展是什么关系？

生态扩展

扩展方式	一句话解释	例子
Skills 技能	一段教智能体"怎么做某事"的说明书 + 配套脚本	EduHub 社区技能, <code>deeptutor skill install</code> 安装（有安全门）
MCP 服务器	标准协议的外部工具箱，接上就能用	你在 <code>mcp_servers</code> 项目里学的那些服务器
CLI Apps	把本机命令行程序包装成工具给智能体用	内置 101 个，如 <code>ffmpeg</code> 、 <code>python</code>

三者最终都汇成一件事：**给 Agent 循环的工具列表里加条目**。这和你学的 **[LLM工具调用]** 完全同构——扩展 DeepTutor = 给循环发明新工具。

Q10

它和我已学的 langchain Agent 是什么关系？我为什么要读它的源码？

知识串联

你在 langchain 项目里亲手搭过最小 **[[Agent循环]]**（两节点状态图）；DeepTutor 就是这个循环的 **工业化成品**，在每一环上都做了生产级加固：

环节	langchain 实验	DeepTutor
循环引擎	LangGraph 两节点图	自研 loop + 轮次预算 + 崩溃可恢复的流式会话
工具协议	@tool 装饰器	BaseTool/ToolDefinition + 按能力动态挂载
记忆	无	L1/L2/L3 三层 + Memory Graph
检索	单一 VectorStore	五引擎可插拔 + 索引版本化
自我纠错	参数校验回传（ [[质检Gate与自我纠错循环]] ）	同类机制 + 连接探测、嵌入失配检测、doctor 体检

读源码的性价比最高的三个文件： capabilities/subagent/tools.py（60 行看懂子智能体机制）、services/memory/paths.py 开头（20 行看懂三层记忆布局）、services/rag/pipelines/ 目录名（一眼看懂引擎清单）。

Q11

它给我什么能力？我能拿它干什么？

能力清单

系统提供的能力：

结合你水平的具体用法：

- 啃教材：**上传 PDF/EPUB，逐章精读 + 页码级引用问答
- 出题批改：**Quiz 题库 + Mastery Path 按掌握度闯关
- 研究报告：**Deep Research 自动检索联网写报告
- 可视化：**讲解中的概念直接生成交互图表/动画
- 视频学习：**贴 YouTube 链接，时间戳级提问
- 外挂大脑：**把本机 Claude Code 当解题/动手外挂
- 把 400+ 项目清单变成一门"课程"，让它出题检验你
- 读 DeepTutor 自己的源码：建一个 KB，问"consult_subagent 的预算在哪强制执行"
- 给 langchain 笔记建库，用 Mastery Path 复盘旧概念
- 研究报告功能调研新项目，产出喂给 vault 入库流程
- 用 Obsidian 连接器直接挂上你的 vault 目录学习

Q12

动动嘴之前再确认一遍：这条链路上每个词我都懂了吗？

术语自查表

术语	一句话解释
Capability 能力	一种学习模式的插件包 = 提示词 + 工具集 + 上下文声明，挂上循环即生效
L1/L2/L3 记忆	原始轨迹 → 界面摘要 → 用户画像，LLM 驱动的逐层压缩，全部可查看可编辑
Knowledge Base	文档经解析、嵌入后建成的可检索库，引擎可插拔、索引带版本
consult_subagent	把本机编程智能体 CLI 当工具调用的那座桥，带每轮咨询预算
Content Workspace	智能体唯一可自由读写的文件夹，产出固定在 outputs/ 下
model_catalog.json	所有模型连接和 key 的家，Settings 页面就是它的编辑器



动手实验（全部命令本机验证过）

实验 A

主实验：从零到和 AI 导师完成第一次对话（5 步）

Step 1 • 安装

```
pip install -U deeptutor
python -m deeptutor --help  ← 验证装好了；若 deeptutor 命令找不到，统一用 python -m deeptutor
```

目的：拿到完整 Web 应用 + CLI。要求 Python 3.11–3.14、Node.js 20+（本机 3.14.7 / v24 通过）。

✅ **预期结果：**末尾出现 Successfully installed deeptutor-1.6.12 ...；--help 打印命令列表。

Step 2 • 初始化配置

```
mkdir my-deeptutor && cd my-deeptutor  # 这个目录就是它的"runtime home"，数据都在 data\ 下
python -m deeptutor init
```

向导 5 问：后端端口（回车=8001）→ 前端端口（回车=3782）→ LLM 供应商（选编号，如 2=Anthropic）→ 是否改 Base URL → 粘 API key → 选模型 → Embedding/搜索可按 **s** 跳过。配置落在 data\user\settings\model_catalog.json（纯 JSON，可随时手改）。

⚠️ **中转站 (API relay) 用户注意：**向导会去 GET {base_url}/models 拉模型列表，多数中转站不支持这个接口且**无超时，会一直卡住**（实测踩坑）。解法：Ctrl+C 跳出后直接手改 model_catalog.json，把 services.llm.profiles[0] 的 binding/base_url/api_key 和 models[0].model 填上即可，重启后 doctor 会告诉你配置是否就绪。

✅ **预期结果：**看到 Settings saved；或手改后运行 python -m deeptutor doctor，LLM 三项检查全 **PASS**。

Step 3 • 启动

```
python -m deeptutor start  # 保持窗口开着：Ctrl+C 同时停掉前后端
```

注意：本机请统一用 python -m deeptutor，**不要用裸命令** deeptutor——pip 把 exe 装到了用户目录 %APPDATA%\Python\Python314\Scripts，它不在 PATH 里，裸命令会直接报“无法识别”（实测踩坑，见下方坑 4）。

✅ **预期结果：**浏览器打开 <http://127.0.0.1:3782> 看到聊天界面；新开终端 curl <http://127.0.0.1:8001/docs> 返回 200。

Step 4 • 第一问（命令行版）

```
python -m deeptutor run chat "用一句话解释什么是递归"
```

这是 agent 循环的裸入口，能最清楚地看到循环元数据。

✅ **预期结果：**得到中文回答，末尾一行元数据—— capability=chat rounds=1 tokens=8.1k cost=\$0.0016（实测数字）。**cost 是真实成本，看着它会心疼，也就学会了省 token。**

Step 5 • 体检

```
python -m deeptutor doctor
```

✅ **预期结果：**表格显示 LLM 配置/凭据/端点/存储 全部 PASS，RAG 项在没建库时显示 SKIP。

实验 B

副实验：本地嵌入 + 建第一个知识库（4 步，体验完整 RAG）

Step 1 • 拉一个嵌入模型

```
ollama pull nomic-embed-text    # 274MB, 本地跑嵌入, 不花钱不外传
```

⚠ **踩坑实录：**若报 `redirect target ... resolves to non-public 198.18.x.x`, 是代理软件的 fake-ip DNS 把下载域名解析成了假 IP, Ollama 拒绝跳转。解法（无需管理员）：从 `registry.ollama.ai/v2/library/nomic-embed-text/manifests/latest` 拿 manifest, 用 curl 把每个 blob 下到 `~\.ollama\models\blobs\sha256-<hash>`, manifest 存到 `manifests\registry.ollama.ai\library\nomic-embed-text\latest`, `ollama list` 即可见。

✅ **预期结果：**`ollama list` 列出 `nomic-embed-text:latest` 274 MB。

Step 2 • 在 Settings 里配嵌入

Settings → Providers（或直接改 `model_catalog.json`）：provider 选 **Ollama**, endpoint 用默认 `http://localhost:11434/api/embed`, 模型 `nomic-embed-text`, 维度 768。

Step 3 • 建库

```
python -m deeptutor kb create python-basics --doc 你的笔记.md    # 支持 pdf/docx/md 等
```

✅ **预期结果：**`Creating KB python-basics ... via LlamaIndex... Knowledge base created successfully.`

Step 4 • 检索验证

```
python -m deeptutor kb search python-basics "深拷贝和浅拷贝的区别是什么"
```

✅ **预期结果：**返回笔记原文的相关章节（实测精确命中“浅拷贝只复制第一层...”段落）—— 这就是 RAG 的“开卷查资料”。

实验 C

副实验：把本机 Claude Code 接进来当外挂（3 步）

Step 1 • 确认被认出

前提：本机装了 Claude Code（`claude` 命令可用）。打开 Web 界面 → Settings → My Agents，应看到 Claude Code 已被**自动发现**；确认后默认 `permission_mode=bypassPermissions`（无头运行不会卡在授权确认）。

Step 2 • 咨询一次

聊天界面选择该子智能体后提问，或命令行：

```
python -m deeptutor run chat --tool subagent "咨询 Claude Code：解释一下你刚才用了哪些工具"
```

Step 3 • 观察三件事

✅ **预期结果**：① 末尾元数据 `tools=1 rounds=2`（真的调了一次工具）；② 子智能体的中间步骤在侧边栏**实时可见**；③ 问工作区外的文件路径时它会明确说“够不到”——**亲眼看到权限边界生效**，比读十遍文档都有用。

本机踩坑实录（中文 Windows 必看）

坑 1 • 系统代理把 localhost 也劫持了

开着 Clash 类代理时，DeepTutor 访问本机 Ollama 会得到 502 Bad Gateway。解法：命令行前加 `NO_PROXY=localhost,127.0.0.1`（浏览器一般不走代理，不受影响）。

坑 2 • init 向导在自定义 Base URL 时卡死

向导的“拉取模型列表”对不支持的端点无超时。中转站用户直接手改 `model_catalog.json`（结构见实验 A Step 2）。

坑 3 • Ollama 拉模型被 fake-ip DNS 卡住

198.18.x.x non-public 错误 → 手动下载 blob 方案见实验 B Step 1。

坑 4 • deeptutor 命令找不到 (本机已修复)

pip 装到用户目录时 exe 在 %APPDATA%\Python\Python314\Scripts, 不在 PATH, 裸命令 deeptutor start 直接报"无法识别"。两条路: ① 统一用 `python -m deeptutor ...` 最稳; ② 把该 Scripts 目录加进用户 PATH (2026-10-01 已加, 新开终端生效)。改完配置记得: **start 要在 runtime home 目录里跑, 且窗口保持开着。**